



Design And Analysis of Algorithms

Unit 3: 2-3 Trees

Dr. Divyashree N
Department of Computer Science and Engineering

Design And Analysis of Algorithms

2-3 Trees

Topics Covered

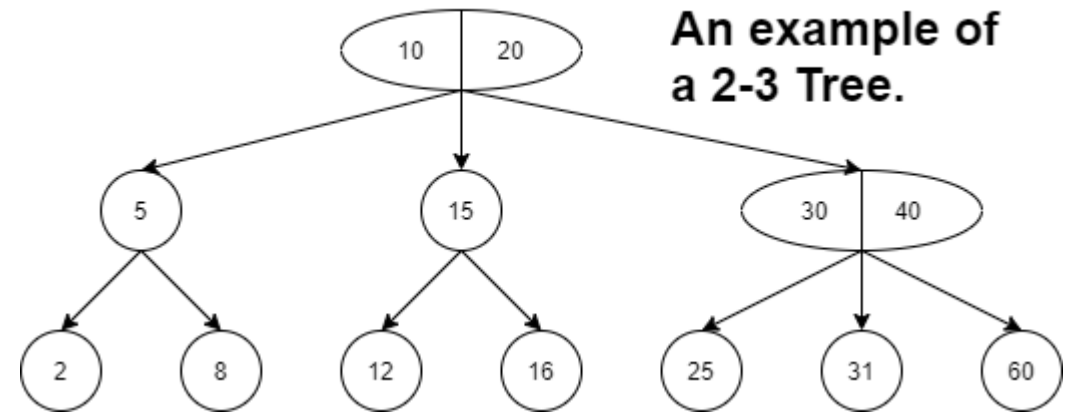
- Introduction to Tree Data Structures
- Definition of 2-3 Trees
- Properties of 2-3 Trees
- Structure of 2-3 Nodes
- Searching in 2-3 Trees
- Insertion in 2-3 Trees
- Deletion in 2-3 Trees
- Balancing Mechanism
- Comparison with Other Trees
- Advantages of 2-3 Trees
- Disadvantages of 2-3 Trees

Introduction to 2-3 Trees

A 2-3 Tree is a **tree data structure** where every node with children has either two children and one data element or three children and two data elements.

Ensures balance by maintaining all leaves at the same depth.

A special case of B-trees with order 3.



Properties of 2-3 Trees

A 2-3 tree follows the below mentioned properties.

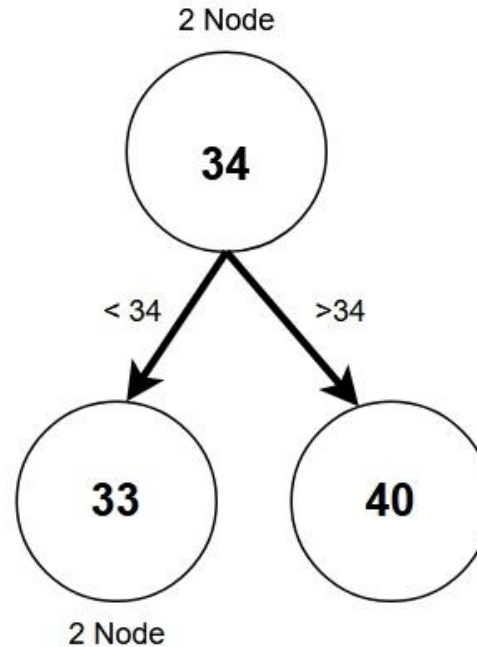
1. Every internal node in the tree is a **2-node** or a **3-node** i.e it has either one value or two values.
2. A node with one value is either a leaf node or has exactly two children. **Values in left sub tree < value in node < values in right sub tree.**
3. A node with two values is either a leaf node or has exactly 3 children. It cannot have 2 children. **Values in left sub tree < first value in node < values in middle sub tree < second value in node < value in right sub tree.**
4. All leaf nodes are at the same level.

Note: A 4-NODE, with three data elements, may be temporarily created during manipulation of the tree but is never persistently stored in the tree.

Structure of 2-3 Nodes

2-node:

- Exactly two children.
- Left child with smaller weight.
- Right child with greater weight.
- Can be a leaf node with no child.



3-node

Exactly three children.

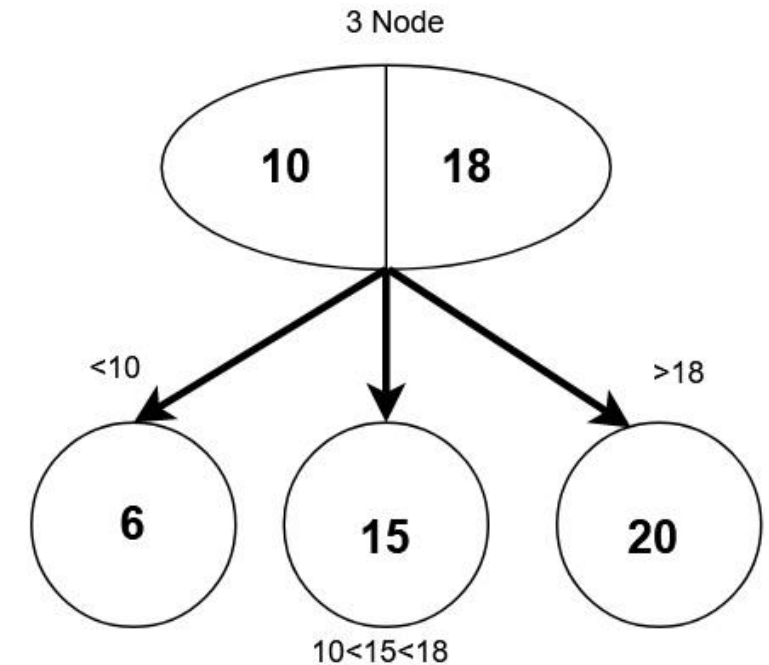
2 data values.

Left child with weight smaller than left data value.

Middle child with weight in between both data values.

Right child with weight greater than right data value.

Can never be a leaf node.



Searching in 2-3 Trees

1. Similar to the search operation in a binary search tree as the data is sorted.
 2. To search X in a 2-3 tree.
 3. If tree is empty → return False
 4. If we reach root node then return False (not found)
 5. If X is less than the left data part, then search the left subtree
 6. If X is greater than left data and more than right data, search the middle subtree.
 7. If X is greater than the right data part, then search right subtree.
-
- Efficiency: Time complexity is $O(\log N)$

Unit 3: 2-3 Trees

Searching in 2-3 Trees

Algorithm for search:

Function Search(node, key):

 If node is NULL:

 Return "Key not found"

 If key == node.key1 OR (node has two keys AND key == node.key2):

 Return "Key found"

 If key < node.key1:

 Return Search(node.left, key) // Go to left child

 Else if node has two keys AND key > node.key2:

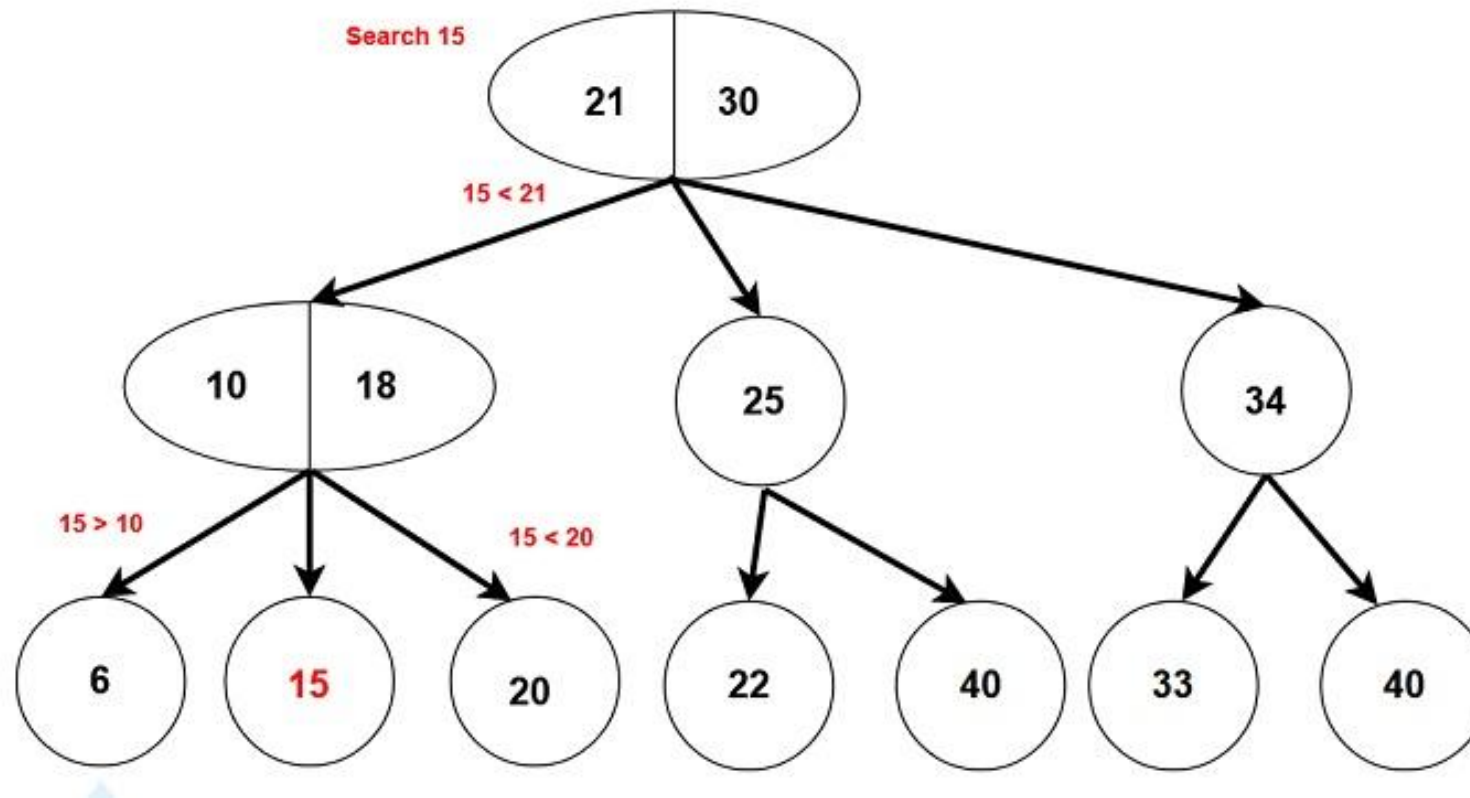
 Return Search(node.right, key) // Go to right child

 Else:

 Return Search(node.middle, key) // Go to middle child

Searching in 2-3 Trees

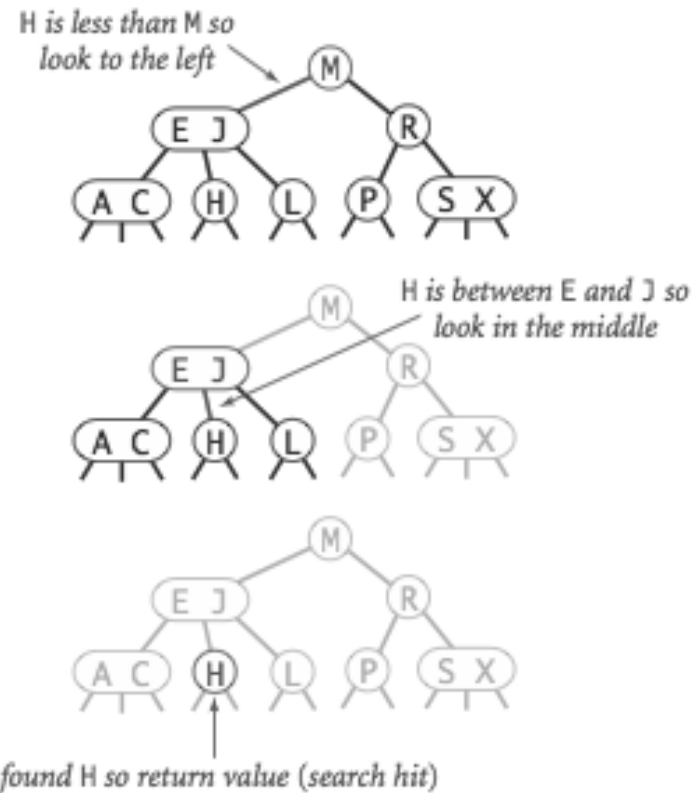
Let's Try it out with an example:



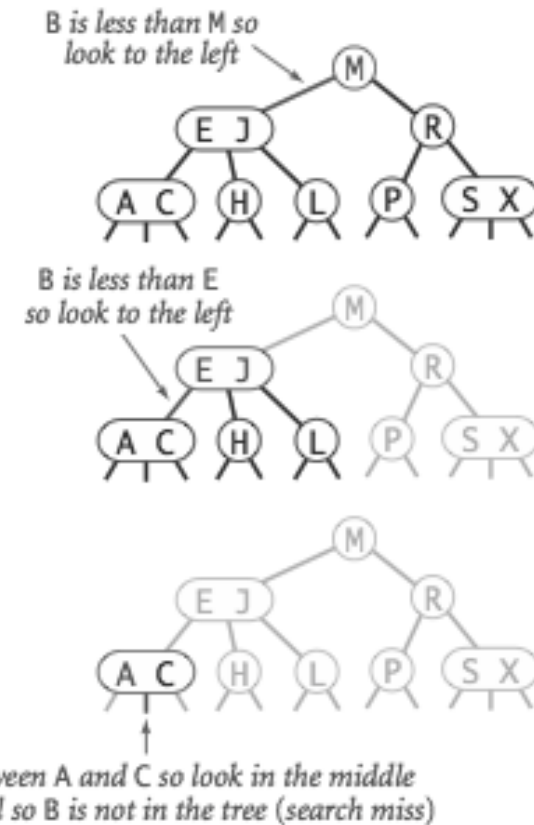
Searching in 2-3 Trees

Let's try this one here:

successful search for H



unsuccessful search for B

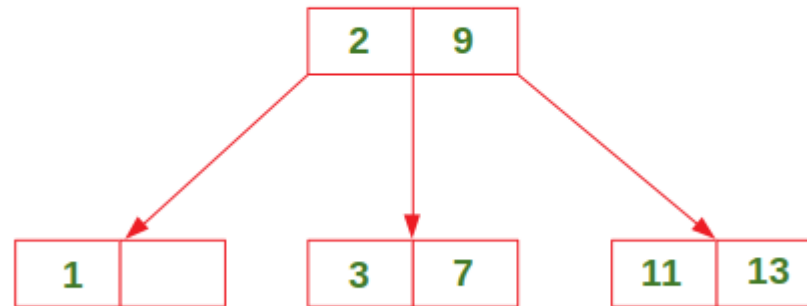


Search hit (left) and search miss (right) in a 2-3 tree

Searching in 2-3 Trees

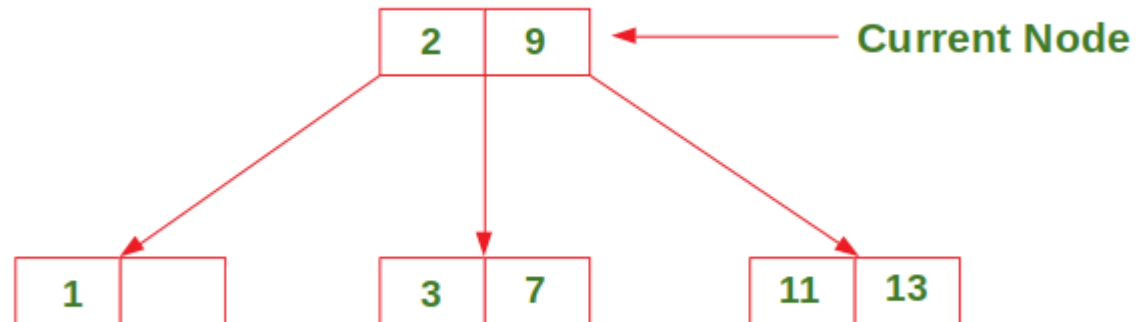
An example of an unsuccessful search:

Search 5 in the following 2-3 Tree:



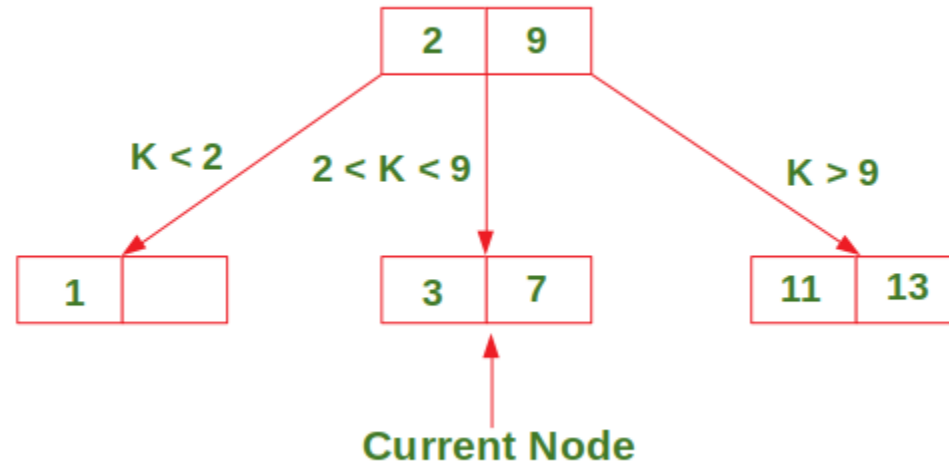
Searching in 2-3 Trees

An example of an unsuccessful search:



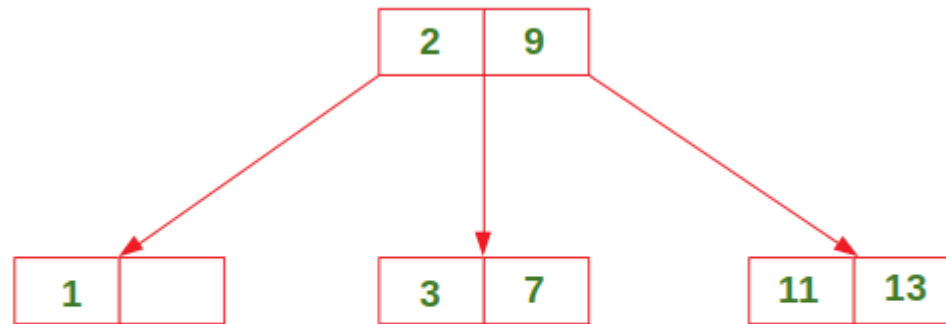
Searching in 2-3 Trees

An example of an unsuccessful search:



Searching in 2-3 Trees

An example of an unsuccessful search:



5 Not Found. Return False

Insertion in 2-3 Trees

- To insert X in a 2-3 tree.
- If the tree is empty → Add X as root.
- Else: Search for the right position of X and add it as a leaf node.
- If there is a leaf node with one data part, add X with and leaf node becomes 2 - Node.
- If the leaf node has two data parts, add X. Split temporary 3-nodes and move data to parent nodes according to the sorting order.

Insertion in 2-3 Trees

Algorithm for insertion:

Function Insert(node, key):

 If node is a leaf:

 Insert key into node in sorted order

 If node now has three keys:

 Return Split(node)

 Else:

 Return node

 If $\text{key} < \text{node.key1}$:

 newChild, newKey = Insert(node.left, key)

 Else if node has two keys AND $\text{key} > \text{node.key2}$:

 newChild, newKey = Insert(node.right, key)

 Else:

 newChild, newKey = Insert(node.middle, key)

 If newChild is not NULL:

 Insert newKey into node in sorted order

 Attach newChild to the correct position

 If node now has three keys:

 Return Split(node)

 Return node, NULL

Function Split(node):

 Create newNode

 newNode.key1 = node.key2

 If node has children:

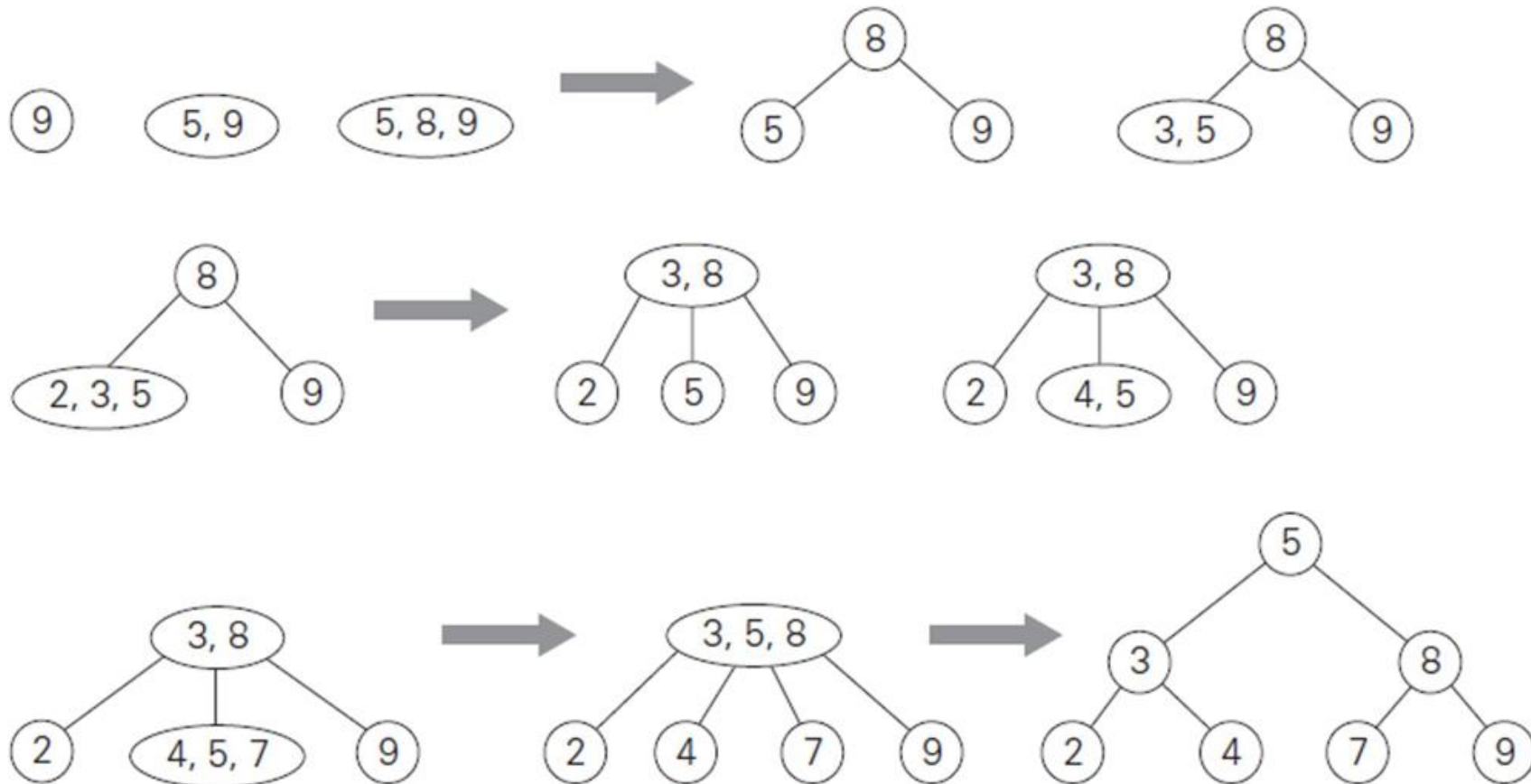
 newNode.left = node.middle

 newNode.right = node.right

 Return newNode, node.key1 // Promote middle key

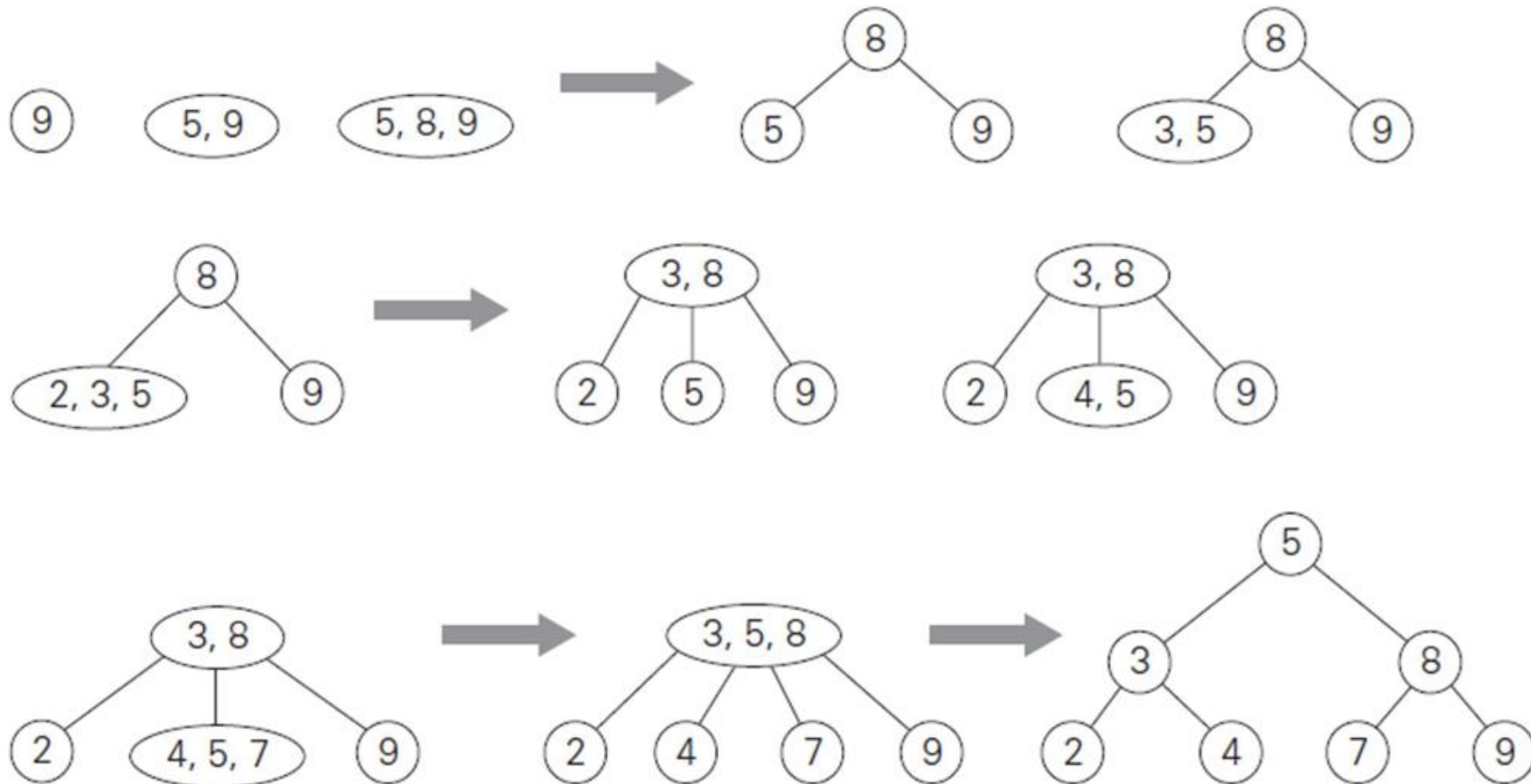
Insertion in 2-3 Trees

Let us insert 9, 5, 8, 3, 2, 4, 7 starting from an empty tree.



Insertion in 2-3 Trees

Let us insert 9, 5, 8, 3, 2, 4, 7 starting from an empty tree.



Deletion in 2-3 Trees

To delete X in a 2-3 tree.

If the tree is empty return false.

Else: Search for the position of X and delete it, then adjust the tree.

If X is part of 3 node delete X and adjust left value and middle value. Also adjust node's ancestor's left and middle value if necessary.

If X is part of 2 node then adjust and split the tree in a recursive manner arranging nodes in sorted order.

Deletion in 2-3 Trees

Algorithm for Delete:

Function Delete(node, key):

//Case 1:

 If node is NULL:

 Return NULL

 If key is in a leaf node:

 Remove key from node

 If node has no keys left:

 HandleUnderflow(node)

 Return node

Deletion in 2-3 Trees

Algorithm for Delete:

Function Delete(node, key):

// Case 2:

 If key is in an internal node:

 inorderSuccessor = FindMin(node.right)

 node.key1 = inorderSuccessor

 Delete(node.right, inorderSuccessor)

 Else if key < node.key1:

 Delete(node.left, key)

 Else if node has two keys AND key > node.key2:

 Delete(node.right, key)

 Else:

 Delete(node.middle, key)

 If node has underflow:

 HandleUnderflow(node)

 Return node

Deletion in 2-3 Trees

Algorithm for Delete:

Function Delete(node, key):

//Case 3:

Function HandleUnderflow(node):

 If sibling has extra key:

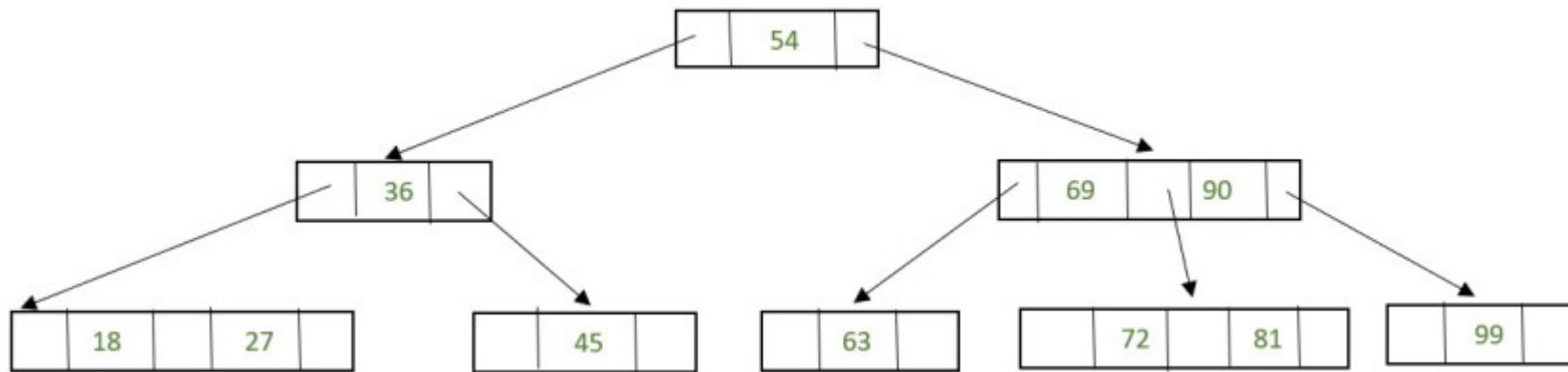
 Borrow key from sibling (Redistribution)

 Else:

 Merge node with sibling

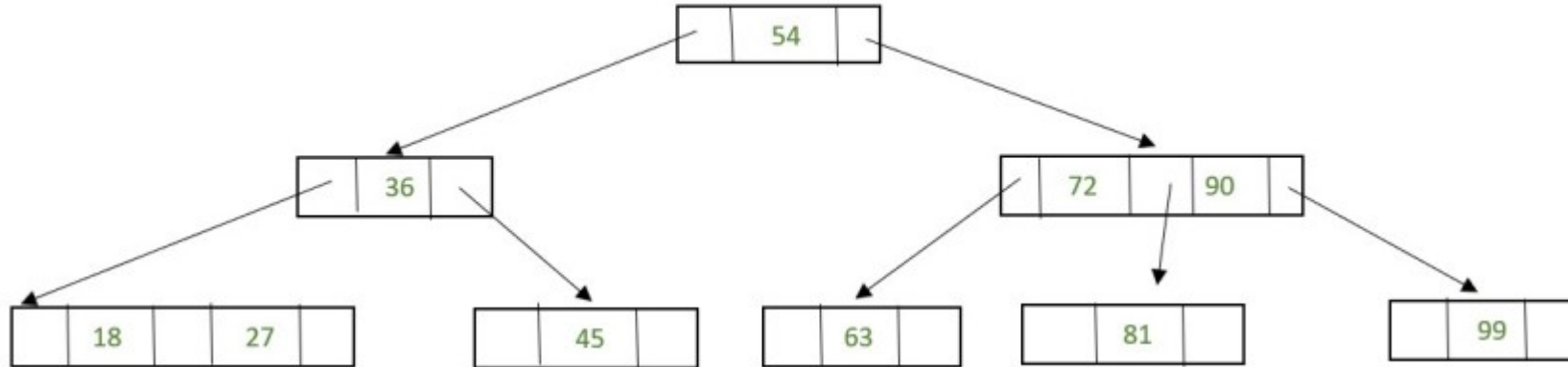
Deletion in 2-3 Trees

Consider the 2-3 tree given below : delete the following values from it: 69, 72, 99, 81.



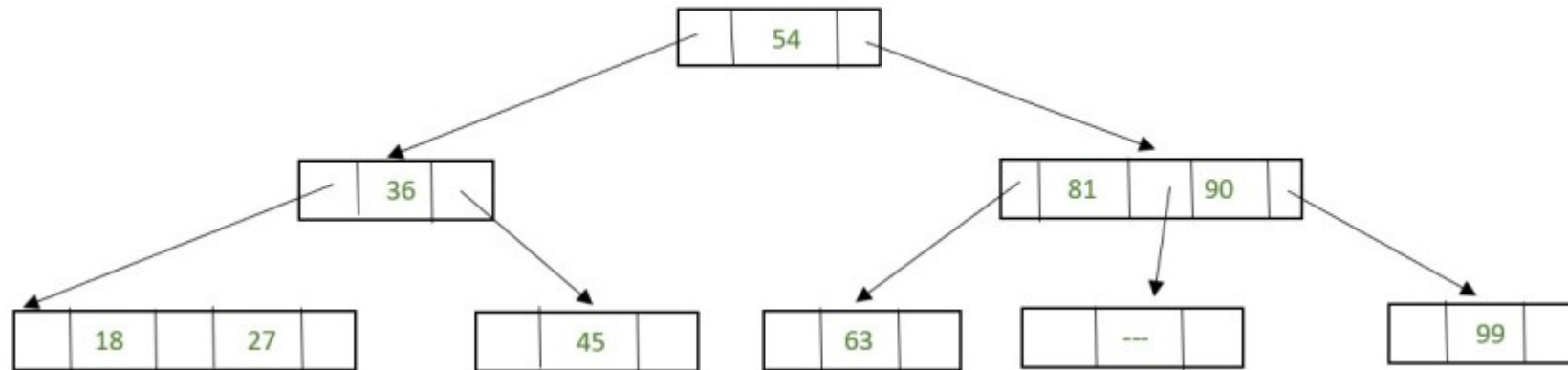
Deletion in 2-3 Trees

To delete 69, swap it with its in-order successor, that is, 72. 69 now comes in the leaf node. Remove the value 69 from the leaf node.



Deletion in 2-3 Trees

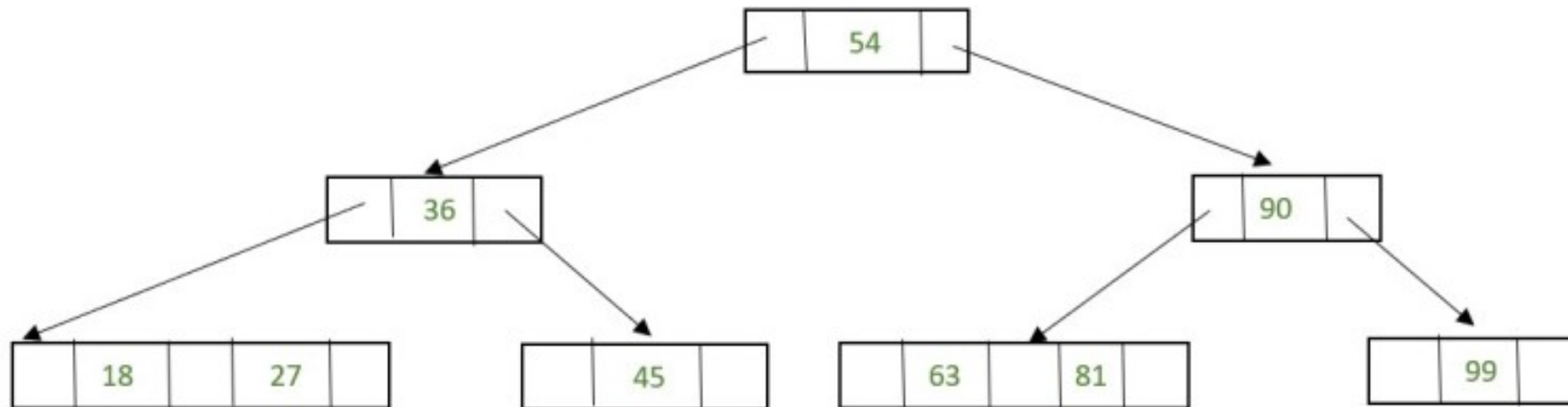
To delete 72, 72 is an internal node. To delete this value swap 72 with its in-order successor 81 so that 72 now becomes a leaf node. Remove the value 72 from the leaf node.



Deletion in 2-3 Trees

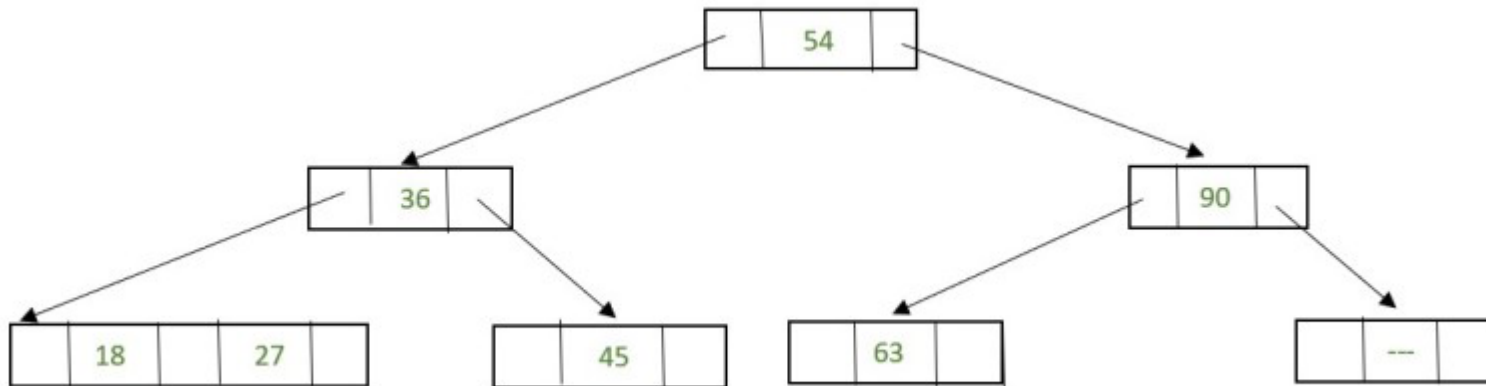
Now there is a leaf node that has less than 1 data value thereby violating the property of a 2-3 tree. So the node must be merged.

To merge the node, pull down the lowest data value in the parent's node and merge it with its left sibling.



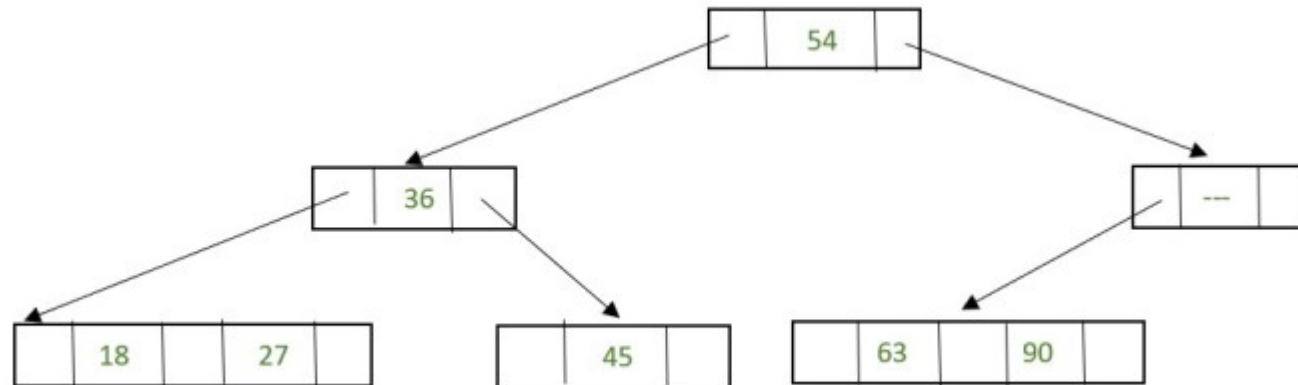
Deletion in 2-3 Trees

To delete 81, 81 is an internal node. To delete this value swap 81 with its in-order successor 90 so that 81 now becomes a leaf node. Remove the value 81 from the leaf node.



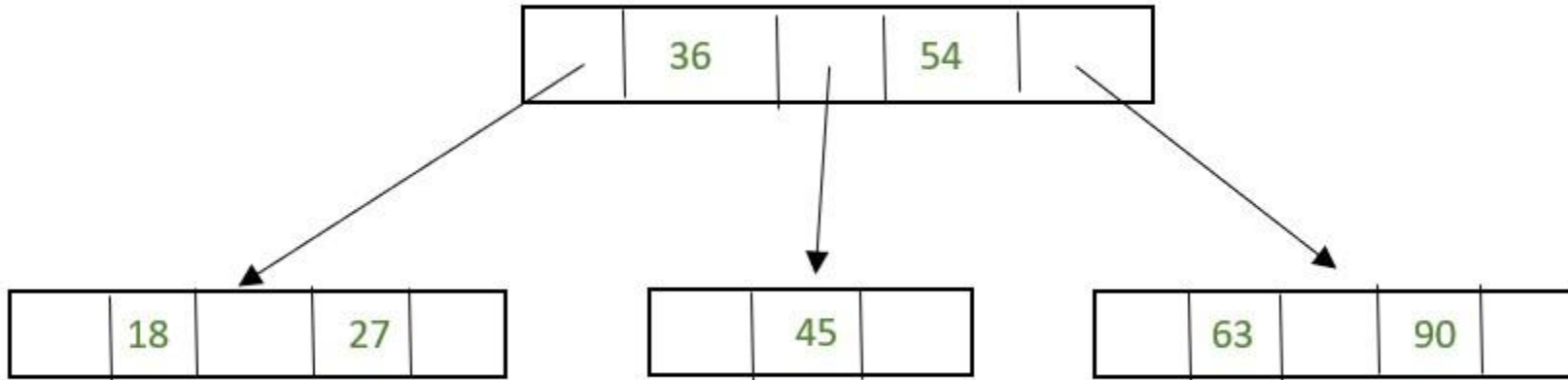
Deletion in 2-3 Trees

Now there is a leaf node that has less than 1 data value, thereby violating the property of a 2-3 tree. So the node must be merged. To merge the node, pull down the lowest data value in the parent's node and merge it with its left sibling.



Deletion in 2-3 Trees

As internal node cannot be empty. So now pull down the lowest data value from the parent's node and merge the empty node with its left sibling.



The Balancing Mechanism

The Key to a 2-3 Trees' efficiency is its ability to have a search time complexity of $O(\log(n))$.

This efficiency is maintained, as the tree is re-balanced for each insertion or deletion.

Here's how Balance is Maintained:

- Node splitting during insertion.
- Key redistribution or merging during deletion.

Comparison with Other Trees

Feature	Binary Search Tree	AVL Tree	Red-Black Tree	2-3 Tree
Balancing Mechanism	None	Rotations	Rotations	Splitting/Merging
Time Complexity	$O(N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
Use Case	Simple applications	In-memory sorting	Complex dynamic sets	Database indexing

Merits and Demerits

Advantages of 2-3 Trees

1. Strictly balanced structure ensures predictable performance.
2. Guaranteed $O(\log N)$ time complexity for search, insert, and delete operations.

Disadvantages of 2-3 Trees

1. More complex to implement compared to simpler structures like BSTs or AVL trees.
2. Overhead in maintaining balance during frequent insertions or deletions.

References

1. <https://www.geeksforgeeks.org/2-3-trees-search-and-insert/> (step by step guide for search, insertion and deletion)
2. <https://iq.opengenus.org/2-3-trees/> (understanding tree properties)
3. <https://www.tutorialspoint.com/2-3-trees-data-structures-and-algorithms-in-cplusplus> (illustration)



PES
UNIVERSITY

CELEBRATING 50 YEARS

THANK YOU

Teaching Associate
Varun Bharadwaj